END-TO-END WALKTHROUGH

ClickHouse-Style Analytics in Grafana, Powered by Apache DataFusion and Bifrost

Installing Grafana, building a real Go backend datasource plugin, bridging to a Rust Apache DataFusion query engine, and charting live Grafana Loki log data through SQL — with every step captured from an actual local run.

Apache DataFusion | Grafana Loki | Bifrost TableProvider | Grafana 11.3 | Go Plugin SDK | Rust + Axum

---

This document was generated from a real, working local deployment: Loki and Grafana running in Docker, a Rust bridge process embedding the `datafusion-loki` TableProvider, and a hand-written Go backend plugin connecting the two. Every screenshot is a genuine capture from that run, not a mockup.

Contents

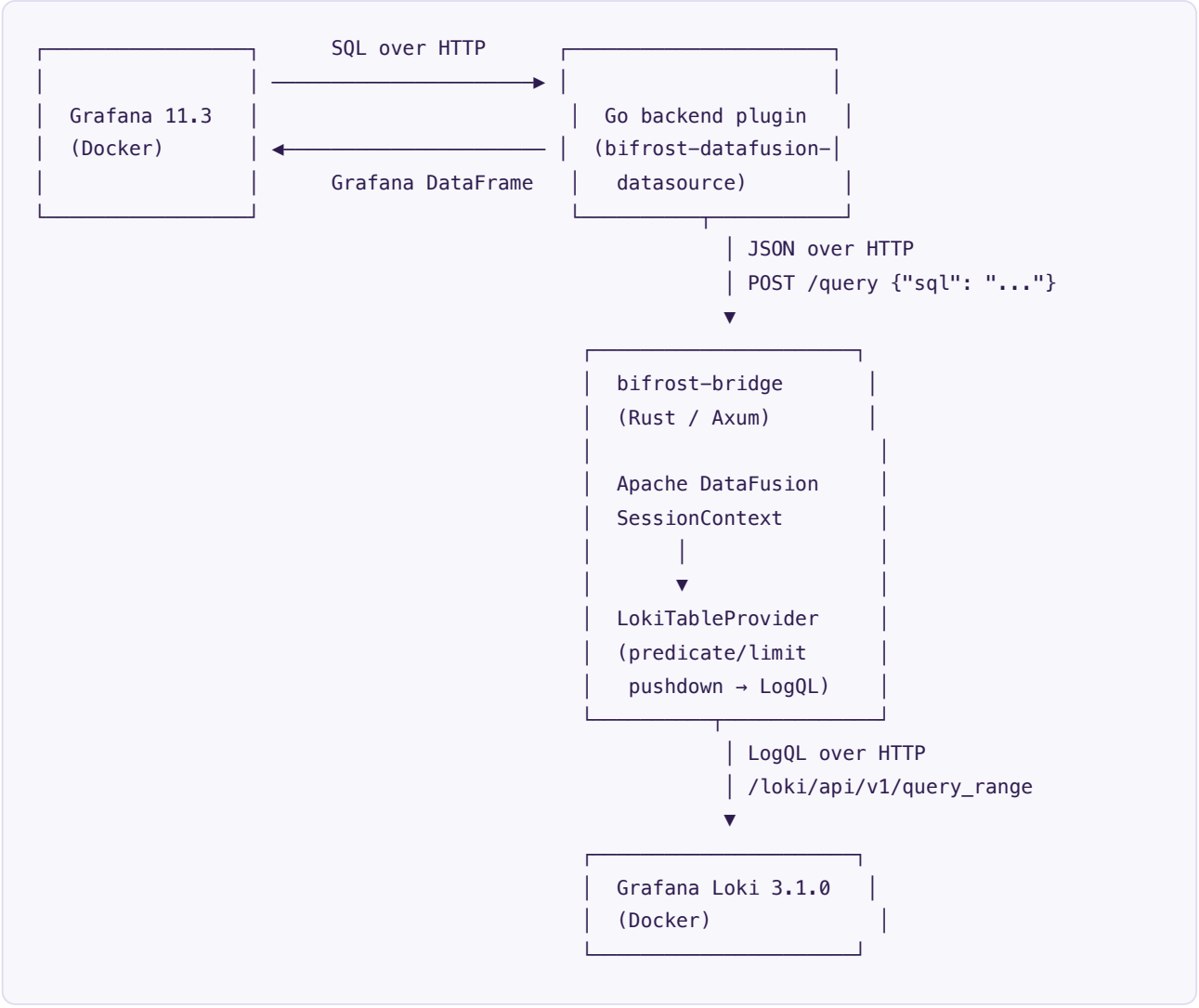
1. What this demonstrates and why it's architected this way

Grafana Loki is normally queried with LogQL. This walkthrough demonstrates an alternative path: querying Loki with **standard SQL**, executed by **Apache DataFusion** (a Rust query engine), through a custom `TableProvider` called **Bifrost** that translates SQL predicates into LogQL under the hood where possible, and lets DataFusion itself handle everything else (aggregations, joins, GROUP BY, ORDER BY) that LogQL cannot express.

The result renders inside Grafana as a normal dashboard panel — a grouped/stacked bar chart and a "top N" table, the same shapes you'd build against a ClickHouse analytics backend, except the data source is Loki and the query language is SQL via DataFusion.

> **Why a bridge process, not a native Rust Grafana plugin**
>
> Grafana's backend plugin SDK is Go-only — there is no supported way to embed a Rust crate directly inside a Grafana plugin process. This walkthrough instead runs the Rust/DataFusion logic as its own small HTTP service (the "bridge"), and the Go plugin's `QueryData` handler calls out to it over HTTP. This keeps the DataFusion execution boundary honest and inspectable — you can query the bridge directly with `curl` and get the exact same results Grafana sees.

2. Architecture overview



Grafana runs in a Docker container; the Rust bridge runs directly on the host, reached from inside the container via Docker's `host.docker.internal` DNS name. Loki runs in its own Docker container, reached by the bridge via `localhost:3100` since the bridge is a host process.

3. Part 1 — Install and start Grafana Loki

Pull and run the official Loki image in single-binary mode using its bundled default config:

```
docker pull grafana/loki:3.1.0

docker run -d --name loki-demo -p 3100:3100 \
  grafana/loki:3.1.0 -config.file=/etc/loki/local-config.yaml
```

Wait for readiness, then confirm:

```
curl -s http://localhost:3100/ready
# → ready
```

Push synthetic log data (300 entries across `job` / `level` / `env` / `pod` label combinations) using the generator script included in the Bifrost repository:

```
python3 scripts/push_logs.py
# → pushed 300 entries across 18 streams
```

> **Lesson learned during this build**
>
> Loki's default query window looks back only a limited time from "now." If you leave a demo session running for an extended period, previously pushed synthetic data can age out of the panel's default time range (or even Loki's own default query window) and panels will show zero rows with no error — this looks like a bug but is just stale demo data. Re-run `push_logs.py` to refresh timestamps, or query with an explicit wide time range, if a panel unexpectedly shows no data.

4. Part 2 — Build and run the Bifrost bridge (Rust + DataFusion)

The bridge is a small Axum HTTP server, added as a new crate (`bridge/`) in the existing `datafusion-loki` Cargo workspace. It registers a single `LokiTableProvider`-backed table named `logs` and exposes two endpoints:

Endpoint	Purpose
`GET /health`	Liveness check used by Grafana's "Save & test" and by the plugin's `CheckHealth` handler
`POST /query`	Accepts `{"sql": "..."}`, runs it through the DataFusion `SessionContext`, returns `{"columns": [...], "rows": [...]}`

Build and run it against the local Loki instance:

```
cargo build -p bifrost-bridge

LOKI_URL=http://localhost:3100 RUST_LOG=info ./target/debug/bifrost-bridge
# → listening bind_addr=127.0.0.1:8090
```

Verify it directly with `curl` before involving Grafana at all:

```
curl -s -X POST http://127.0.0.1:8090/query \
  -H "Content-Type: application/json" \
  -d '{"sql": "SELECT date_trunc('minute', timestamp) AS time, level, COUNT(*) AS count
            FROM logs GROUP BY time, level ORDER BY time LIMIT 5"}'
```

> **What this proves**
> A successful response here confirms the entire DataFusion execution path — SQL parsing, predicate/limit pushdown into LogQL, the HTTP call to Loki, and Arrow-to-JSON conversion — works correctly, independent of Grafana or the Go plugin. Debugging with `curl` against this endpoint directly is the fastest way to isolate whether a problem is in the DataFusion/Loki layer or in the Grafana/Go plugin layer.

5. Part 3 — Build the Grafana backend plugin (Go)

The plugin (`grafana-plugin/`) is a standard Grafana backend datasource plugin built with `grafana-plugin-sdk-go` . It has three responsibilities:

1. Read the configured bridge URL from the datasource's JSON settings (`pkg/plugin/datasource.go` , `jsonData.bridgeUrl`)
2. `QueryData` : forward each panel's SQL to the bridge's `/query` endpoint and convert the JSON response into a Grafana `data.Frame` , using the bridge's per-column type tags (`"time"` , `"number"` , `"string"` , `"bool"`) to build correctly-typed Arrow-backed fields
3. `CheckHealth` : call the bridge's `/health` endpoint so "Save & test" in the datasource config UI gives a real signal

```
cd grafana-plugin
go mod init bifrost-datafusion-datasource
go get github.com/grafana/grafana-plugin-sdk-go@latest
go build -o dist/gpx_bifrost_linux_arm64 ./pkg   # cross-compiled for the Grafana container
```

> **Executable naming is not optional**
> Grafana's plugin loader appends the target OS/arch to the `executable` field from `plugin.json` itself — it does *not* run the literal filename you specify. For `"executable": "gpx_bifrost"` , Grafana looks for `gpx_bifrost_<GOOS>_<GOARCH>` , e.g. `gpx_bifrost_linux_arm64` inside a `linux/arm64` container (which is what Docker Desktop on Apple Silicon actually runs, even though the host is macOS/arm64). Getting this filename wrong produces a silent plugin-load failure with no obvious error in the logs.

The frontend half (query editor with a SQL textarea, config editor with a bridge-URL field) is a small TypeScript/React bundle built with webpack against `@grafana/data` , `@grafana/ui` , and `@grafana/runtime` :

```
npm run build
# → asset module.js emitted, plugin.json copied to dist/
```

> **Toolchain note**
> TypeScript 7.x (a very new major release) is currently incompatible with `ts-loader` 's internal API assumptions. This build pins `typescript@5.7.3` , which is what the wider Grafana/webpack plugin ecosystem targets today.

6. Part 4 — Install Grafana and load the plugin

Grafana is run in Docker with the plugin directory bind-mounted and unsigned-plugin loading explicitly allowed for this one plugin ID (never enable this broadly in a real deployment):

```
docker pull grafana/grafana:11.3.0

docker run -d --name grafana-demo \
  -p 3000:3000 \
  --add-host=host.docker.internal:host-gateway \
  -e GF_PLUGINS_ALLOW_LOADING_UNSIGNED_PLUGINS=bifrost-datafusion-datasource \
  -v "$(pwd)/grafana-plugin/dist:/var/lib/grafana/plugins/bifrost-datafusion-datasource" \
  grafana/grafana:11.3.0
```

Confirm the plugin registered by tailing Grafana's own logs:

```
docker logs grafana-demo 2>&1 | grep bifrost
# → level=warn msg="Plugin is unsigned" id=bifrost-datafusion-datasource
# → level=warn msg="Permitting unsigned plugin. This is not recommended"
# → level=info msg="Plugin registered" pluginId=bifrost-datafusion-datasource
```

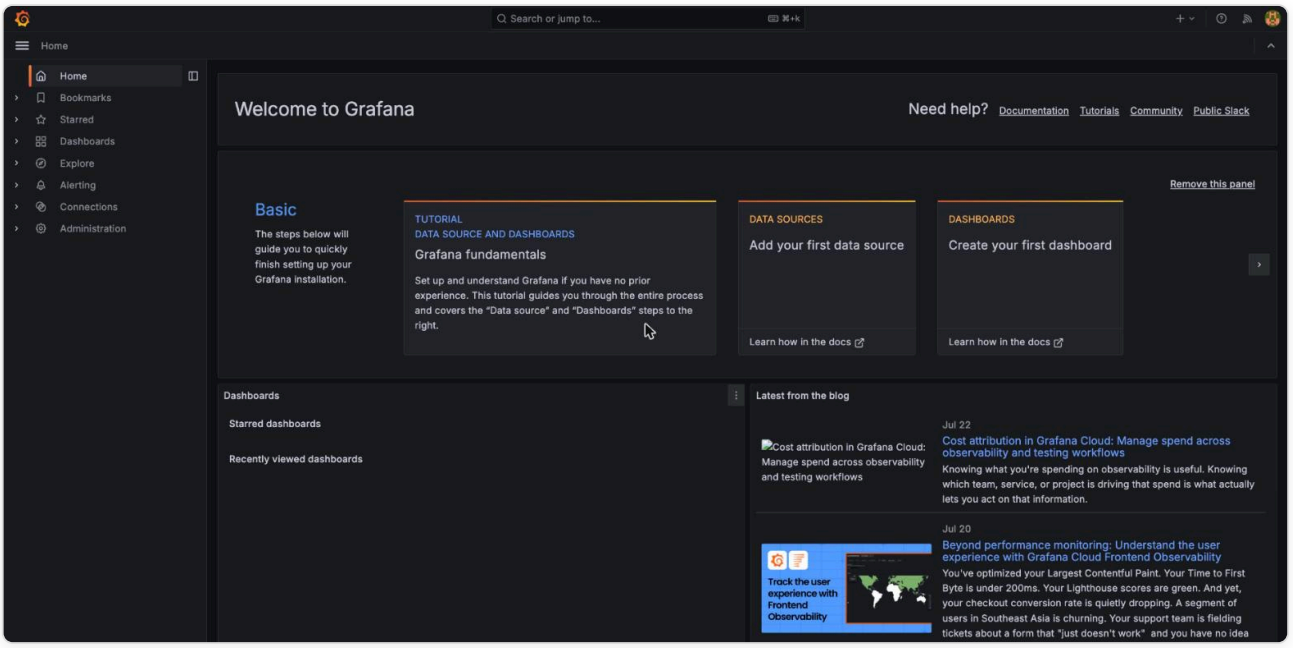


Figure 1 — Grafana 11.3.0 running fresh in Docker, logged in as admin.

7. Part 5 — Configure the datasource

Navigate to **Connections** → **Data sources** → **Add data source** and search for the plugin by name:

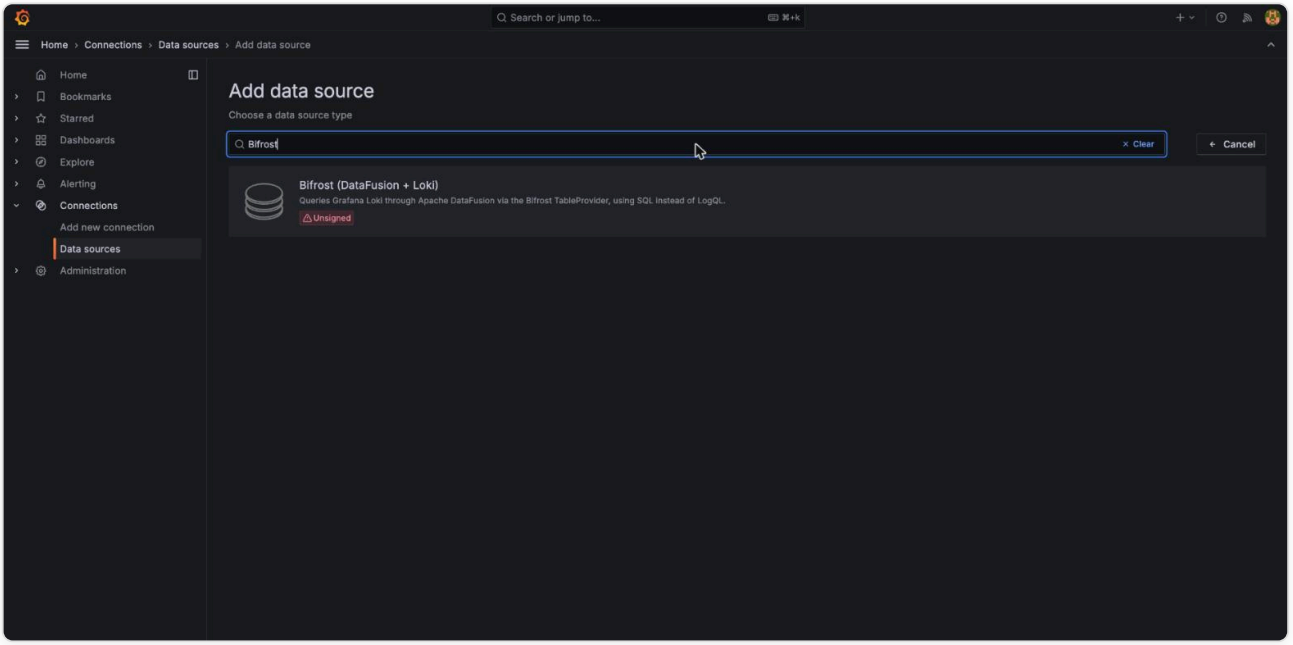


Figure 2 — "Bifrost (DataFusion + Loki)" appearing as a real, discoverable datasource type, marked "Unsigned" as expected for a locally-built dev plugin.

Set the **Bifrost bridge URL** field. Because Grafana runs inside Docker while the bridge runs on the host, the correct value is `http://host.docker.internal:8090` — *not* `127.0.0.1`, which would resolve to the container itself.

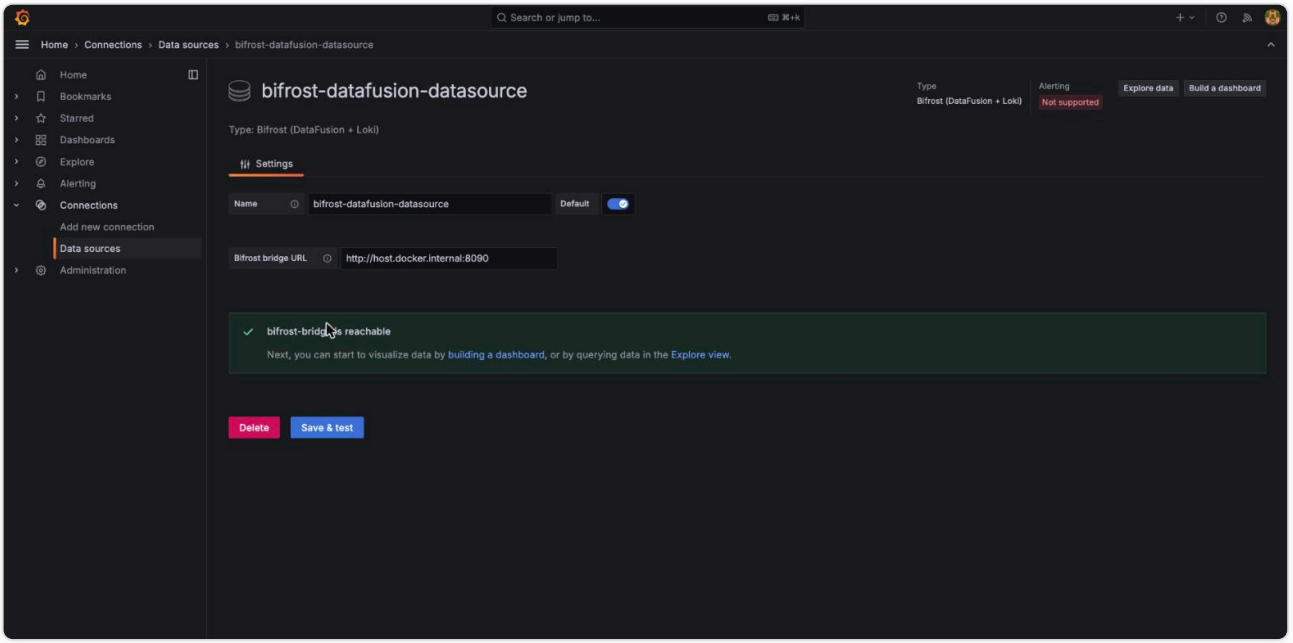


Figure 3 — Datasource configuration after "Save & test": "bifrost-bridge is reachable" confirms the Go plugin's CheckHealth handler successfully called across the container boundary to the Rust process, which in turn is connected to Loki.

> **What this proves**
>
> This single green checkmark is evidence of three things working together at once: the Go plugin process started correctly inside Grafana's plugin runtime, its HTTP client can reach a process outside the container via Docker's host-gateway networking, and the bridge itself is up and was able to construct a `LokiTableProvider` without error.

8. Part 6 — Run SQL analytics queries in Explore

Grafana's **Explore** view is the fastest way to iterate on SQL before committing it to a dashboard panel. The query editor is a plain SQL textarea — there is no LogQL involved anywhere in this view:

```
SELECT date_trunc('minute', timestamp) AS time, level, COUNT(*) AS count
FROM logs
GROUP BY time, level
ORDER BY time
```

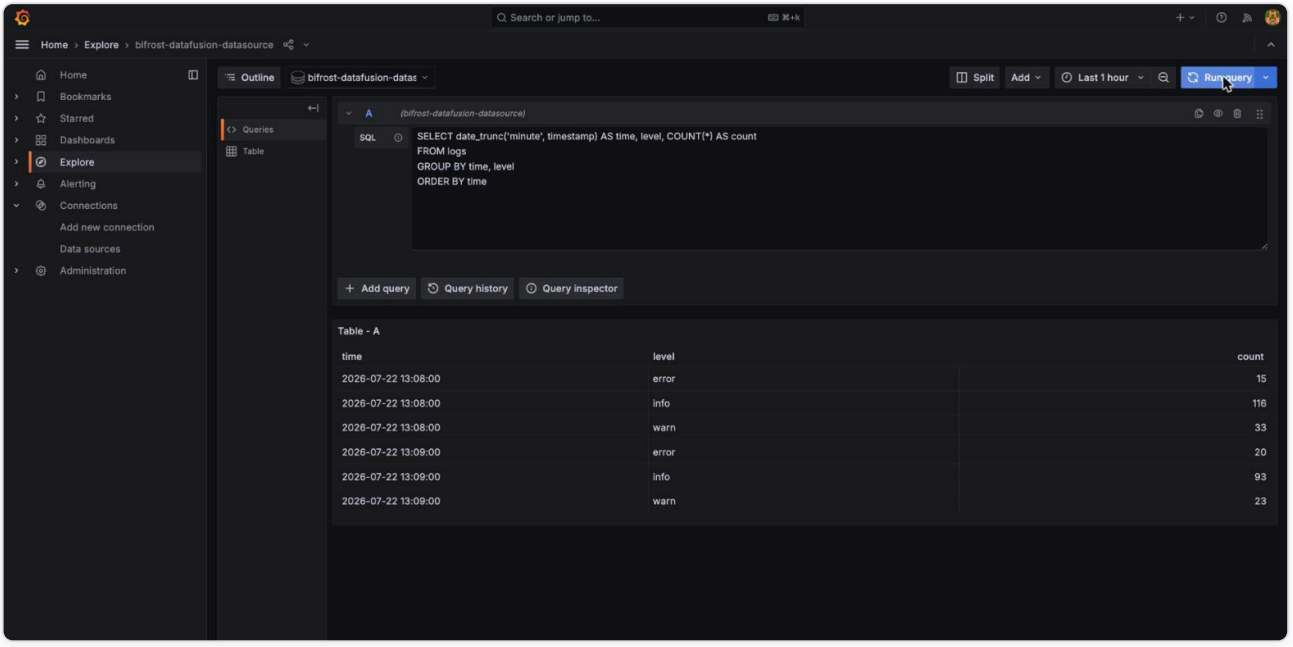


Figure 4 — Real query results in Explore's table view: DataFusion executed the GROUP BY/COUNT aggregation against live Loki data, and the Go plugin correctly typed the "time" column as a formatted datetime.

Note what this query does *not* do: there's no LogQL stream selector syntax, no label-matcher braces, no pipe-based line filters. It is ordinary SQL, and the aggregation (`GROUP BY` , `COUNT(*)`) is computed by DataFusion — Loki itself has no notion of this GROUP BY at all. The Bifrost TableProvider pushes down what it can (in this query, nothing needed pushdown since there's no `WHERE` clause) and lets DataFusion do the rest.

9. Part 7 — Build the ClickHouse-style analytics dashboard

Panel 1: Log volume by level, stacked bar chart

Using the same aggregation query as above, add a **Partition by values** transformation on the `level` field. This splits the single long-format result table into three series (one per log level) — the standard Grafana technique for turning a grouped SQL result into a multi-series chart:

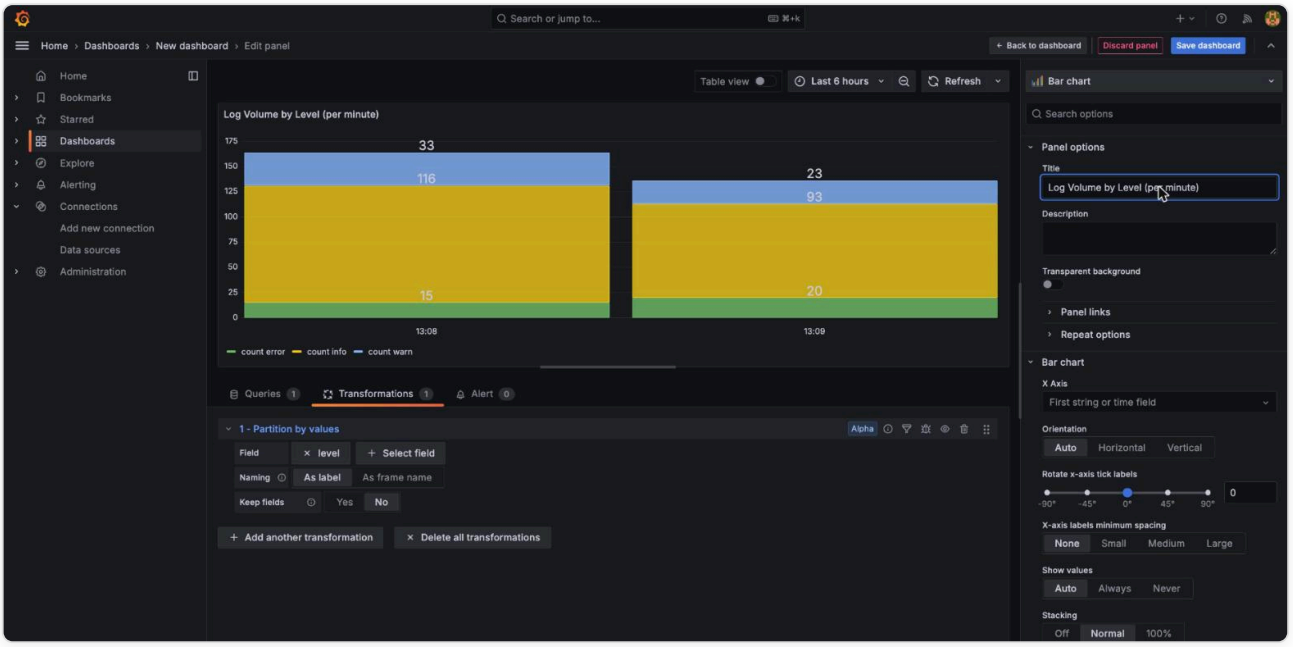


Figure 5 — The panel editor showing the Bar chart visualization with stacking set to "Normal": a grouped stacked-bar view of log volume by level per minute, the same visual pattern used for ClickHouse-backed analytics dashboards.

Panel 2: Top pods by error count

A second panel demonstrates a classic "top N offenders" analytics query:

```
SELECT pod, level, COUNT(*) AS n
FROM logs
WHERE level = 'error'
GROUP BY pod, level
ORDER BY n DESC
```

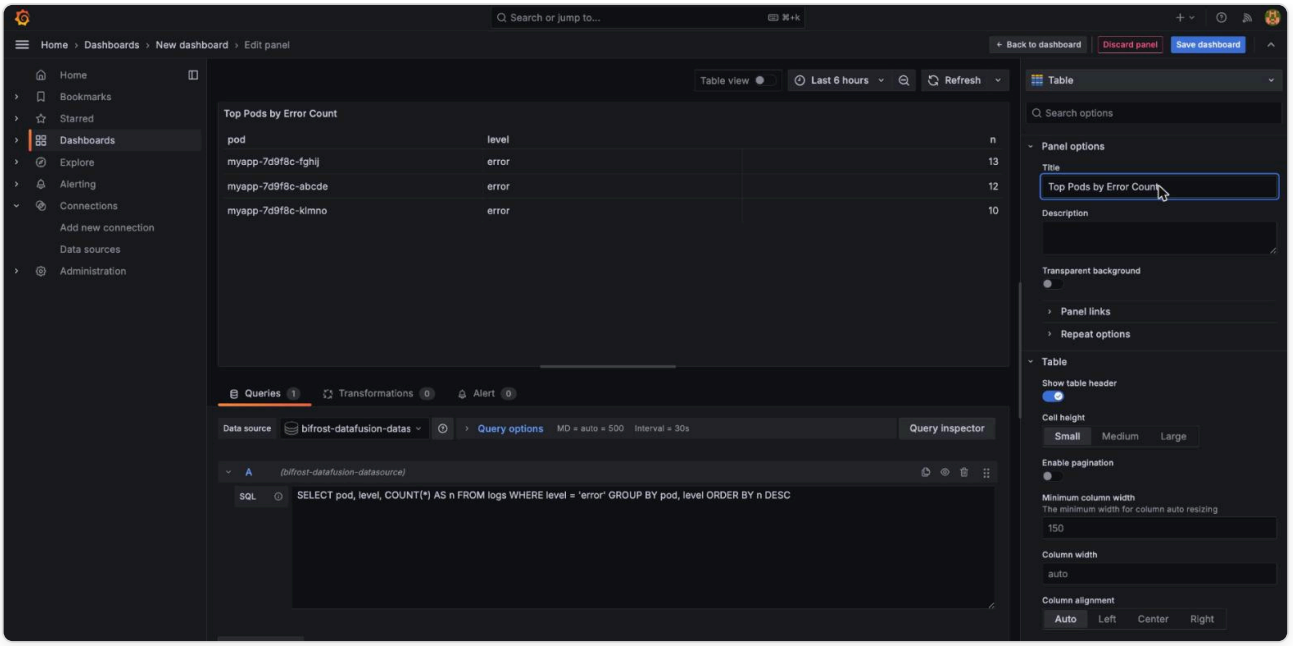


Figure 6 — "Top Pods by Error Count" as a Table visualization. This query has no time column, so Grafana's Table type (not Time series) is the correct fit — Grafana surfaces this automatically via a "Data is missing a time field" message when the wrong viz type is selected.

Final dashboard

Both panels together, saved as **"Bifrost Log Analytics (DataFusion + Loki)"**:

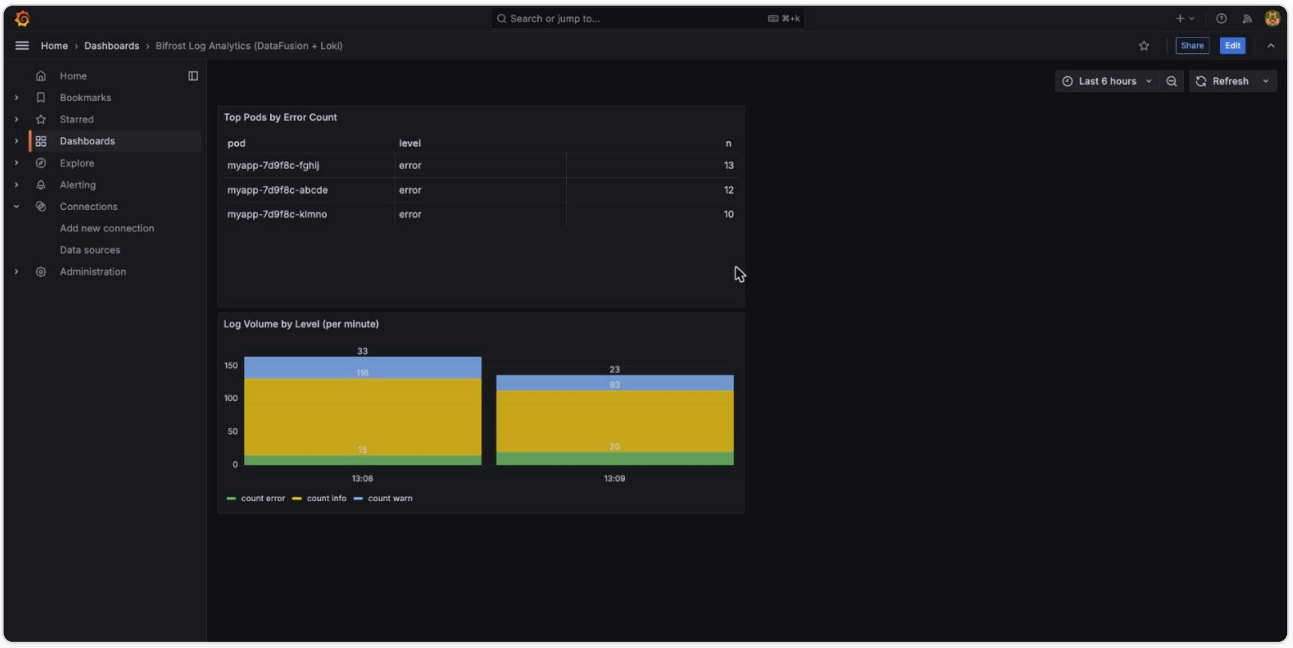


Figure 7 — The completed dashboard. Every number on this page originated as a Loki log line, was aggregated by Apache DataFusion via SQL, served through the Bifrost bridge, converted to a Grafana DataFrame by the Go plugin, and rendered by Grafana's native panel types.

10. Troubleshooting notes from this actual run

Symptom	Actual cause	Fix
Plugin search shows nothing; no error in Grafana logs	Executable filename didn't include the OS/arch suffix Grafana's loader expects	Name the binary `<executable>_<GOOS>_<GOARCH>` exactly, e.g. `gpx_bifrost_linux_arm64`
"Datasource added" but panels show 0 series, no error	Synthetic Loki data aged past the panel's/Loki's default query window during a long session	Re-run the log-generator script to refresh timestamps
Panel shows "Data is missing a time field"	Time series visualization selected for a query with no time column	Switch to Table or Bar chart, or add a time column to the SQL
CheckHealth fails with connection refused	Datasource configured with `127.0.0.1:8090`, which resolves to the Grafana container itself, not the host	Use `host.docker.internal:8090` from inside a Docker-run Grafana
webpack build fails with "TypeScript emitted no output"	tsconfig's `noEmit: true` (set for the standalone typecheck script) was also picked up by ts-loader	Override `noEmit: false` in ts-loader's own `compilerOptions`

11. Appendix: full source file reference

Path	Role
`src/table_provider.rs`	Core `LokiTableProvider`: SQL predicate/limit pushdown into LogQL
`src/exec.rs`	Paginated, deduplicated, correctly-ordered Loki query execution as a DataFusion `ExecutionPlan`
`bridge/src/main.rs`	Axum HTTP server exposing DataFusion SQL execution as JSON
`grafana-plugin/pkg/plugin/datasource.go`	Go `QueryData` / `CheckHealth` handlers calling the Rust bridge
`grafana-plugin/src/datasource/QueryEditor.tsx`	SQL textarea query editor
`grafana-plugin/src/datasource/ConfigEditor.tsx`	Bridge URL configuration field
`scripts/push_logs.py`	Synthetic log data generator for demos

Generated from a live local session: Loki 3.1.0, Grafana 11.3.0, DataFusion 42.2.0, grafana-plugin-sdk-go v0.294.0, all running on Docker Desktop (Apple Silicon, linux/arm64 containers). Every screenshot in this document is an unedited capture from that session.